

Avances del Proyecto — Sistema de IA Anticipatoria para el STC Metro

Alumno: Alan Bellon García **Asesor:** M. en Fil. C. Enrique Francisco Soto Astorga **Fecha de este reporte:** 2026-09-27 (actualizado — ver [Sección 4](#))

Este documento resume el estado técnico y metodológico del prototipo descrito en el anexo de titulación ("Modelado y prototipado de un sistema de Inteligencia Artificial Anticipatoria para la estimación de estados de congestión en el STC Metro de la Ciudad de México"), con base en el código y los datos actualmente presentes en `00_Programas/tesis_metro_ai/`.

1. Mapeo contra los objetivos de la tesis

Objetivo secundario (Anexo)	Estado	Evidencia en código
Generar dataset sintético de afluencia/disrupciones	✅ Completo (14 días sintéticos, laboral+fin de semana, eventos estocásticos)	<code>generador_sintetico_horario.py</code> , <code>simulador_congestion.py</code> , <code>datos_procesados/*.csv</code>
Representar la red como grafo con pesos	✅ Completo	<code>grafo_metro.py</code> → <code>grafo_base_metro.gexf</code>
Modelo de estimación a horizonte corto (10–60 min)	⚠️ Parcial (horizonte discreto de 1 hora, no continuo 10-60 min)	<code>entrenador_anticipatorio.py</code> → <code>modelos/modelo_anticipatorio_rf.pkl</code>
Algoritmo de ruteo que integre la métrica predictiva	✅ Prueba de concepto funcional	<code>ruteo_anticipatorio.py</code>
Integración estimación + ruteo en prototipo funcional	✅ Funcional en un caso (Pantitlán → Auditorio, 7:00); con eventos estocásticos ya no hay desvío en ese caso	<code>ruteo_anticipatorio.py</code>
Explicabilidad de las recomendaciones	✅ Iniciado (importancia de variables)	<code>entrenador_anticipatorio.py</code> → <code>importancia_variables.png</code>
Sistema reactivo de comparación (índice 5.5)	✅ Implementado como baseline dentro del mismo script	<code>ruteo_anticipatorio.py</code> (ruta estática vs. ruta IA)

En términos del índice tentativo, el proyecto ha cubierto el **capítulo 4** completo (modelado y generación de datos) y tiene un **primer corte funcional del capítulo 5** (diseño del prototipo: arquitectura, modelo predictivo, ruteo, baseline reactivo). Falta evaluación formal de KPIs (5.4), profundizar explicabilidad (6.5) y ejecutar comparativas sistemáticas (6.4) sobre múltiples escenarios, no solo el caso de prueba único.

2. Pipeline técnico actual (orden de ejecución)



2.1 `grafo_metro.py` — Modelado topológico

- Filtra GTFS masivo del Valle de México por `agency_id == "METRO"` para aislar rutas, viajes y `stop_times`.
- Construye grafo no dirigido `G(V,E,W)` con **195 nodos** (andenes/estaciones).
- Peso estático de cada arista = promedio de `departure_sec → arrival_sec` entre paradas consecutivas del mismo `trip_id`, filtrando deltas ilógicos (`<0` o `>1800s`).
- Agrega aristas de **transbordo peatonal** (5 min fijos) entre andenes que comparten `stop_name`, habilitando cambios de línea en el grafo.
- Exporta a `grafo_base_metro.gexf` (formato estándar, reutilizable por NetworkX).

Limitación notable: el penalti de transbordo es una constante (5 min) igual para todas las estaciones de correspondencia, sin distinguir estaciones con transbordos largos conocidos (p. ej. Ciudad Azteca, La Raza, Tacubaya).

2.2 `generador_sintetico_horario.py` — Motor de demanda

- Limpia encoding (latin1 → utf-8) de estación/línea provenientes del portal de datos abiertos.

- Clasifica estaciones en 3 perfiles espaciotemporales (`origen` , `destino` , `mixto`) mediante listas fijas (hardcoded) de 10 estaciones por extremo.
- Desagrega afluencia diaria a horaria multiplicando por curvas bimodales (`peso_origen` , `peso_destino` , `peso_mixto`) calibradas manualmente para 24 horas.
- **Matriz Origen-Destino sintética** vía modelo gravitacional simplificado: calcula `atractividad_destino` según hora y perfil, normaliza por estación de origen y reparte las entradas reales observadas como viajes probables a cada destino.
- **[Actualizado 2026-09-27]** Parametrizado por `DIAS_SIMULACION` (lista de fechas, hoy 14 días de enero 2026). Cada fecha se etiqueta `laboral` / `fin_de_semana` según su día de la semana; el fin de semana usa un perfil horario derivado programáticamente del laboral (aplanado de picos + desplazamiento de actividad a mediodía/tarde), no una segunda tabla de constantes inventada. Genera una `matriz_od_sintetica_<fecha>.csv` por día y un `manifiesto_dias_simulados.csv` (fecha, tipo_dia, archivo) que `simulador_congestion.py` consume para no duplicar la lista de días entre scripts.

2.3 `simulador_congestion.py` — Motor de estrés dinámico

- Carga el grafo base y la matriz O-D del día simulado.
- Por cada hora de operación (5–23h), copia el grafo, enruta cada flujo O-D con `nx.shortest_path` (ponderado por `tiempo_minutos`) y acumula `carga_pasajeros` por arista.
- Aplica **función BPR adaptada** para convertir carga en tiempo congestionado:

$$T_c = T_b \times (1 + \alpha \cdot (V/C)^\beta), \alpha=0.15, \beta=4, \text{ tope en } 4 \times T_b$$

con `CAPACIDAD_PROMEDIO_TRAMO_HORA = 35000` (constante única para toda la red).

- **[Actualizado 2026-09-27] Generador de eventos estocásticos** (reemplaza la inyección determinista de Línea 9): por cada (hora, línea, tipo de evento) del día se sortea ocurrencia con un proceso de Poisson (tasa base modulada por hora pico y, para lluvia, por temporada de lluvias), y si ocurre, severidad (Beta(2,5), sesgada a eventos leves) y duración (1-4 horas). `lluvia` afecta toda una línea de superficie ($A/B/12$); `falla_mecanica` e `incidente_plataforma` afectan un tramo puntual de cualquier línea. El efecto se sigue expresando como carga fantasma sobre la fórmula BPR (no se escribe en `carga_pasajeros_red` , que queda como ridership real limpio), y además se registra explícitamente en el dataset vía `hay_evento` / `tipo_evento` / `severidad_evento` .
- El ruteo ya no recalcula `nx.shortest_path` por cada fila de la matriz O-D y por cada hora: como el peso de ruteo (`tiempo_minutos`) es estático, la ruta de cada par origen-destino se calcula una sola vez y se cachea (necesario para que escalar a 14 días fuera viable en tiempo razonable: ~5.5 min totales en vez de ~14× el tiempo de un solo día).
- Construye ventanas temporales (t-1, t, t+1) de `congestibilidad` por tramo (sin fugar información entre días) y exporta `dataset_features_entrenamiento.csv` , ahora con **52,360 registros** de 14 días (antes 3,740 de un solo día).

2.4 `entrenador_anticipatorio.py` — Modelo predictivo

- Ordena cronológicamente y separa train/test 80/20 **sin aleatorizar** (evita fuga de información temporal).
- Features: `hora`, `tiempo_ideal`, `congestibilidad_t`, `congestibilidad_t_minus_1` y, **[2026-09-27]** cuando el dataset trae contexto de eventos: `hay_evento`, `severidad_evento` y dummies one-hot de `tipo_evento` (necesarias porque `RandomForestRegressor` no acepta texto). Target: `target_congestibilidad_t_plus_1`.
- `RandomForestRegressor` (150 árboles, profundidad 10) evaluado con MAE/RMSE.
- Genera gráfica de importancia de variables (`importancia_variables.png`) como primer paso de explicabilidad.
- Exporta modelo a `modelos/modelo_anticipatorio_rf.pkl`.

2.5 `ruteo_anticipatorio.py` — Integración estimación + ruteo

- Carga grafo base + modelo entrenado + dataset de contexto (usado como proxy de "sensores en tiempo real").
- `crear_grafo_futuro(fecha, hora)`: para cada arista con datos en esa fecha y hora, predice el retraso futuro y lo suma al tiempo ideal, generando un grafo proyectado. **[Actualizado 2026-09-27]** Las features se toman del propio modelo (`modelo.feature_names_in_`), incluyendo las dummies de `tipo_evento` (las que no aparecen en esa hora se rellenan con 0), para que el ruteo nunca se desincronice del entrenador. El contexto se filtra por fecha además de hora porque el dataset ahora contiene 14 días.
- Compara ruta estática (Dijkstra sobre tiempo ideal) vs. ruta anticipatoria (Dijkstra sobre pesos proyectados por IA), evaluando ambas contra el tráfico real proyectado.
- Caso de prueba demostrado: Pantitlán → Auditorio, 7:00 AM (día laboral 2026-01-13). Con el modelo multi-día, ambos sistemas eligen la misma ruta (Línea 9 hasta Tacubaya, transbordo a Línea 7): tiempo ideal 37.94 min, tiempo proyectado 40.33 min. Ya no se reproduce el desvío que mostraba la versión anterior, porque la falla fija de Línea 9 (7-9am) que lo provocaba fue reemplazada por eventos estocásticos, y ese día a esa hora no hubo un evento que congestionara esta ruta. Para demostrar el beneficio del ruteo anticipatorio hace falta buscar sistemáticamente pares O-D/horas con evento activo (ver Sección 4.4).

3. Limitaciones metodológicas actuales (a atender antes de tesis final)

1. ~~Un solo día de simulación~~ **RESUELTO 2026-09-27** (ver [Sección 4](#)): ahora se simulan 14 días (10 laborales + 4 de fin de semana), y el split 80/20 cronológico separa *días* completos (12 de entrenamiento, 3 de prueba, con 1 día compartido en la frontera del corte), no horas dentro del mismo día.
2. ~~Evento de interrupción hardcodeado~~ **RESUELTO 2026-09-27**: la falla fija de Línea 9 fue reemplazada por un generador de eventos estocásticos (lluvia, falla mecánica, incidente de plataforma) con tasas tipo Poisson por hora/línea/temporada y severidad/duración muestreadas. Queda pendiente calibrar esas tasas contra fuentes reales (ver Sección 4.4).
3. **Capacidad constante para toda la red** (`35000` pasajeros/hora): no diferencia tramos troncales de alta capacidad (Línea 1, 2, 3) de tramos periféricos.

4. **Horizonte de predicción discretizado a 1 hora**, mientras el objetivo de la tesis pide horizonte de 10–60 min. Falta granularidad sub-horaria.
5. **Perfiles origen/destino hardcodedos** (listas fijas de 10 estaciones): no se derivan de un análisis estadístico de la matriz de afluencia real, sino de un supuesto manual razonado.

Ninguno de estos puntos invalida el trabajo — son exactamente el tipo de simplificación esperable en una primera iteración de prototipo — pero deben documentarse como alcance y quedar como candidatos directos para la siguiente fase.

4. Extensión multi-día + eventos estocásticos (2026-09-27)

Esta sección documenta la ejecución del prompt de la sesión anterior (ver historial de commits). Cambios de código:

- `generador_sintetico_horario.py`: parametrizado por `DIAS_SIMULACION` (14 fechas de enero 2026), perfil horario laboral/fin de semana por `tipo_dia`, una matriz O-D por día y un `manifiesto_dias_simulados.csv` de salida. También se restringió el histórico fuente a solo los días simulados (antes desagregaba las ~1,857 fechas del CSV completo a nivel hora sin usarlas después: un archivo intermedio de ~300 MB para un solo día útil).
- `simulador_congestion.py`: lee el manifiesto y simula los 14 días en un solo run; reemplaza la falla fija de Línea 9 por el generador de eventos estocásticos (Poisson por hora/línea/tipo, severidad Beta(2,5), duración 1-4h); cachea rutas estáticas por par origen-destino (el cuello de botella real al escalar a varios días); agrega `tipo_dia`, `hay_evento`, `tipo_evento`, `severidad_evento` al dataset.
- `entrenador_anticipatorio.py`: si el dataset trae columnas de evento, las agrega como features (`hay_evento`, `severidad_evento`, dummies de `tipo_evento`).
- `ruteo_anticipatorio.py`: **corrección detectada al verificar el pipeline completo**. El script seguía construyendo `X_pred` con las 4 features originales, mientras el modelo reentrenado espera 10, y fallaba con `ValueError: The feature names should match those that were passed during fit`. Además tenía un error silencioso: filtraba el contexto solo por `hora`, así que con 14 días cada arista quedaba con el valor del último día leído, mezclando días distintos. Ahora usa `modelo.feature_names_in_` y filtra por `fecha_viaje` + `hora_viaje` (caso de prueba: 2026-01-13, 7:00).

4.1 Dataset resultante

	Versión anterior (1 día, evento determinista)	Versión actual (14 días, eventos estocásticos)
Registros	3,740	52,360
Días distintos	1	14 (10 laborales, 4 fin de semana)
Filas con evento activo	~ desconocido (no etiquetado)	870 (1.66%): 707 lluvia, 94 incidente de plataforma, 69 falla mecánica
Split train/test (80/20 cronológico)	Mismo día en train y test (fuga temporal de facto)	12 días train / 3 días test (1 día compartido en la frontera del corte)
Tiempo de generación (<code>simulador_congestion.py</code>)	segundos	~5.5 min (14 días, con caché de rutas)

4.2 Cambio en MAE / RMSE

Métrica	Baseline (1 día, sin eventos aleatorios)	Nuevo (14 días, eventos estocásticos)	Δ
MAE (min)	0.0000109	0.0028	×256
RMSE (min)	0.000233	0.0313	×134

Lectura correcta de este resultado: el error subió, y eso es lo esperado y lo correcto, no una regresión del modelo. El baseline de 1 día tenía una fuga temporal de facto — el split 80/20 separaba horas del mismo día con el mismo patrón de falla recurrente (L9, 7-9am, todos los días idéntico), así que el modelo memorizaba trivialmente el patrón y el error caía a prácticamente cero ($\text{MAE} \approx 10^{-5}$ min, una precisión sin sentido físico). Con 14 días y eventos verdaderamente estocásticos (distintos tipos, tramos, horas, severidades y duraciones cada día), el modelo debe generalizar de 12 días vistos a 3 días no vistos con eventos nuevos que nunca ocurrieron exactamente igual en entrenamiento. Un MAE de ~0.003 min y RMSE de ~0.031 min sobre un target cuyo rango va de 0 a 6.09 min (media 0.015, con el 75% de los tramos en 0 — la red solo se congestiona en tramos/horas puntuales) es un desempeño razonable para un primer modelo con features todavía limitadas (no incluye, por ejemplo, la línea/tramo como variable categórica). Esto también resuelve la limitación #1 de la Sección 3: el split ahora separa días reales, no horas del mismo día.

Importancia de variables (nuevo modelo): `congestibilidad_t` (50.3%) y `hora` (41.9%) siguen dominando; el bloque de variables de evento (`severidad_evento` , `hay_evento` , dummies de `tipo_evento`) aporta en conjunto ~1.9% — señal débil pero presente, consistente con que solo 1.66% de las filas tienen evento activo. Ampliar esa señal (más días, o sobremuestrear horas con evento) es candidato directo para la siguiente iteración si se quiere que el modelo dependa más del contexto de eventos y menos de la inercia (`congestibilidad_t`).

4.3 Supuestos de simulación explícitos (para citar en el capítulo de metodología)

- Ventana de 14 días naturales de enero 2026 (única franja con datos reales completos en el CSV fuente); no hay quincena ni variación mensual/estacional real en la muestra, solo la distinción laboral/fin de semana.
- Perfil horario de fin de semana derivado programáticamente del laboral (aplanado + desplazamiento a mediodía/tarde), no calibrado con datos reales de fin de semana.

- Tasas de eventos (Poisson), factores de hora pico/temporada de lluvias y factores de impacto por tipo de evento son **ilustrativos**, no una calibración estadística sobre incidencia real del STC (ver Sección "Análisis de factibilidad técnica" más abajo, que ya anticipaba este punto y sigue vigente para la siguiente iteración).

4.4 Siguiente paso

Prompt listo para usar en la siguiente sesión de trabajo:

"El dataset de entrenamiento ya tiene 14 días con eventos estocásticos, pero la señal de evento (`hay_evento` / `tipo_evento` / `severidad_evento`) solo aporta ~1.9% de importancia porque apenas 1.66% de las filas tienen un evento activo. Antes de calibrar tasas contra fuentes reales, sube la proporción de señal explotable: (1) añade `linea` / `tramo` como variable categórica (dummies o target encoding) para que el modelo pueda aprender qué tramos son estructuralmente más propensos a congestión, ya que hoy el modelo no sabe en qué parte de la red está parado; (2) evalúa si migrar de `RandomForestRegressor` a `GradientBoostingRegressor` (o `HistGradientBoosting`, que maneja mejor el desbalance de clases raras) mejora la predicción específicamente en las filas con `hay_evento=1` (repórtalo aparte del MAE/RMSE global, ya que hoy ese segmento es <2% de los datos y puede quedar diluido); (3) documenta en `avances.md` una propuesta concreta de fuentes para calibrar `TASA_BASE_LLUVIA` , `TASA_BASE_FALLA_MECANICA` y `TASA_BASE_INCIDENTE_PLATAFORMA` en `simulador_congestion.py` contra datos reales (climatología SMN/Conagua para lluvia; boletines de contingencia o notas de prensa del Metro para fallas/incidentes), aunque sea como aproximación documentada y no como calibración estadística rigurosa; (4) en `ruteo_anticipatorio.py` , en lugar de un solo caso fijo (Pantitlán → Auditorio, 2026-01-13 7:00, donde hoy la ruta IA coincide con la estática), recorre las fechas/horas del conjunto de prueba con `hay_evento=1` , compara ruta estática vs. anticipatoria para un conjunto de pares O-D que crucen los tramos afectados y reporta en cuántos casos la IA cambia la ruta y cuántos minutos ahorra en promedio."

Análisis de factibilidad técnica: eventos estocásticos (lluvia, contingencias/accidentes)

Nota (2026-09-27): el análisis de esta sección ya fue implementado — ver [Sección 4](#). Se conserva completo porque documenta el razonamiento y las fuentes consideradas para calibrar las tasas de eventos, que sigue siendo trabajo pendiente (Sección 4.4).

Conclusión general: es técnicamente factible y compatible con la arquitectura existente, con esfuerzo moderado. El diseño actual ya separa la "capa de infraestructura" (grafo estático), la "capa de demanda" (matriz O-D) y la "capa de estrés" (función BPR + inyección de carga), lo que da puntos de extensión naturales para insertar estocasticidad sin rediseñar el sistema desde cero.

3.1 Dónde se integra cada tipo de evento

Evento	Efecto físico a modelar	Punto de integración en el código actual
Lluvia	Reduce velocidad de acceso a estaciones de superficie/elevadas (Línea A, B, 12 en tramos), incrementa afluencia por trasvase modal (gente que deja camión/bici por metro)	Multiplificador probabilístico sobre <code>peso_atractividad / pasajeros_viaje</code> en <code>generador_sintetico_horario.py</code> , y/o reducción de <code>CAPACIDAD_PROMEDIO_TRAMO_HORA</code> en tramos de superficie dentro de <code>simulador_congestion.py</code>
Falla mecánica / contingencia de línea	Reduce o anula capacidad de un tramo específico por tiempo variable	Ya existe el mecanismo (inyección de carga fantasma en <code>simulador_congestion.py</code>); solo falta parametrizarlo estocásticamente en vez de hardcodearlo
Accidente/incidente de plataforma	Cierre temporal de estación/tramo (grafo con disponibilidad reducida, no solo más lento)	Requiere nueva función que remueva temporalmente una arista o nodo del grafo (<code>G_hora.remove_edge(...)</code>) en vez de solo penalizar el peso — el grafo actual no maneja indisponibilidad total, solo congestión

3.2 Requisitos técnicos adicionales

- Generador de eventos por proceso estocástico.** Un proceso de Poisson (o Binomial Negativa si se quiere sobredispersión) por hora/línea es suficiente y barato computacionalmente: para cada hora simulada, se sortea si ocurre un evento y, si ocurre, se sortea tipo, tramo afectado, severidad y duración. Esto es perfectamente compatible con el bucle `for hora in horas_operacion` que ya existe en `simulador_congestion.py`.
- Fuente de probabilidades realistas.** Aquí está el reto principal, no el técnico sino el de datos: no hay acceso (según la justificación de la propia tesis) a un feed en tiempo real de incidentes del STC Metro. Opciones viables:
 - Usar reportes históricos públicos (redes sociales oficiales del Metro, prensa, boletines de contingencia) para estimar frecuencia aproximada de incidentes por línea — trabajo de recolección manual/scraping, factible en el tiempo de tesis a escala pequeña (decenas de eventos, no miles).
 - Alternativamente (y es lo recomendable dado el alcance de "prototipo validado con datos sintéticos" que ya declara el anexo), **parametrizar la probabilidad a mano con justificación teórica/documental** (p. ej. "la Línea A reporta X incidentes por trimestre según nota de prensa Y"), dejando explícito que es una aproximación y no una calibración estadística rigurosa. Esto es consistente con el marco metodológico ya aceptado (datos sintéticos, no datos reales en tiempo real).
 - Para lluvia específicamente, sí existe una fuente pública razonable: climatología histórica de la CDMX (SMN/Conagua) para estimar la probabilidad de lluvia fuerte por mes/hora, que es más defendible que inventar una tasa arbitraria.
- Grafo con disponibilidad variable.** El código actual (`grafo_base_metro.gexf`, copiado cada hora) soporta perfectamente eliminar/restaurar aristas por snapshot, ya que cada `G_hora = G_base.copy()` es independiente. No se requiere cambio estructural de librería (sigue siendo NetworkX), solo lógica adicional.

4. **Impacto en el modelo predictivo.** Añadir eventos estocásticos exige ampliar las features del `RandomForestRegressor` (o migrar a un modelo que maneje mejor variables categóricas/exógenas, ej. Gradient Boosting con variables dummy de evento). El riesgo real es de **datos, no de modelo**: con solo 3,740 filas de un día, meter eventos raros (baja frecuencia por diseño) puede dejar clases extremadamente desbalanceadas. Esto refuerza la necesidad del paso previo (varios días simulados) antes de sumar estocasticidad, para tener suficientes muestras positivas de "hubo evento" en el set de entrenamiento.
5. **Costo computacional.** Trivial. Simular $N \text{ días} \times 19 \text{ horas} \times \text{un sorteo de evento por hora}$ es una operación de milisegundos; el cuello de botella sigue siendo el ruteo de la matriz O-D con `nx.shortest_path` por cada par origen-destino, que ya corre hoy sin problema para un día.

3.3 Riesgos y mitigaciones

- **Riesgo:** sobre-ingeniería del generador de eventos distrayendo del objetivo central (ruteo anticipatorio). *Mitigación:* limitar el generador a 2-3 tipos de evento (lluvia, falla de tramo, cierre de estación) con una distribución simple, documentando explícitamente que el objetivo es probar que el sistema **reacciona correctamente ante disponibilidad variable del grafo**, no producir un simulador meteorológico/operativo exhaustivo.
- **Riesgo:** que la probabilidad "inventada" reste rigor metodológico ante el jurado. *Mitigación:* citar fuente para cada parámetro (aunque sea aproximado) y encuadrarlo explícitamente como supuesto de simulación, tal como ya se hace con los perfiles origen/destino y la función BPR actual — el anexo mismo justifica el uso de simulación por la dificultad de acceso a datos reales en tiempo real.

Veredicto: la integración es factible dentro del alcance de la tesis y no requiere nueva infraestructura ni librerías; el trabajo pendiente es principalmente de diseño de parámetros (tasas de ocurrencia) y de expandir el dataset a múltiples días para que el modelo tenga suficientes ejemplos de eventos disruptivos que aprender.

Mensaje de commit sugerido

```
feat: simular 14 dias con eventos estocasticos y reentrenar modelo anticipatorio
```

Reemplaza el dia unico (2026-01-13) y la falla deterministica de Linea 9 por:

- `generador_sintetico_horario.py`: parametrizado por `DIAS_SIMULACION` (14 fechas, laboral/fin de semana), perfil horario de fin de semana derivado del laboral, una matriz O-D por dia y `manifiesto_dias_simulados.csv` como salida.
- `simulador_congestion.py`: generador de eventos estocasticos (lluvia, falla mecanica, incidente de plataforma) via Poisson por hora/linea/temporada, severidad y duracion muestreadas; cache de rutas estaticas por par origen-destino para escalar a multiples dias; agrega `hay_evento/tipo_evento/severidad_evento` al dataset (52,360 filas vs. 3,740 antes).
- `entrenador_anticipatorio.py`: incorpora el contexto de evento como features (dummies de `tipo_evento` + `hay_evento` + `severidad_evento`) cuando estan presentes.

MAE sube de 0.0000109 a 0.0028 min y RMSE de 0.000233 a 0.0313 min frente al dataset de un solo dia: el baseline anterior tenia fuga temporal de facto (mismo patron de falla, split separaba horas del mismo dia). El split ahora separa dias reales (12 train / 3 test) con eventos genuinamente distintos entre si.